

Celebal Technologies Summer Internship Program 2026

Week 5 : Spark Basics Assignment

Submitted By: Manjit Bajaj

Domain: Data Engineering

StudentId: CT_CSI_DE_1160

Objective: Understand Spark fundamentals and perform data cleaning, transformation, and aggregation using DataFrames.

Q1: What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

Ans: Traditional MapReduce relies heavily on disk-based processing. Intermediate results are written to and read from disk, which makes iterative processing slow. It also requires more complex programming using separate Map and Reduce functions.

Apache Spark is preferred because it supports in-memory processing, provides easy-to-use DataFrame APIs, and offers faster processing for iterative and interactive workloads.

Q2: Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

Ans: Spark stores intermediate data in memory whenever possible. In iterative machine learning algorithms, the same data is processed repeatedly.

In disk-based systems such as traditional MapReduce, data must be read from and written to disk during every iteration. Spark keeps the data in memory, reducing disk I/O and making iterative machine learning algorithms significantly faster.

Q3: Write a code snippet to remove all duplicate rows from a DataFrame based on a specific set of columns: user_id and transaction_date.

Ans:

```
df_q3 = df.dropDuplicates(
    ["Transaction ID", "Transaction Date"]
)

df_q3.show(5)
```

✓ 3.7s

Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date
TXN_1000555	Tea	1	1.5	1.5	Credit Card	In-store	2023-10-19
TXN_1001832	Salad	2	5.0	10.0	Cash	Takeaway	UNKNOWN
TXN_1002457	Cookie	5	1.0	5.0	Digital Wallet	Takeaway	2023-09-29
TXN_1003246	Juice	2	3.0	6.0	NULL	NULL	2023-02-15
TXN_1004184	Smoothie	1	4.0	4.0	Credit Card	In-store	2023-05-18

only showing top 5 rows

Duplicate rows were removed based on Transaction ID and Transaction Date, which are the corresponding identifying columns available in the cafe sales dataset.

Q4: Given a DataFrame df_sales, write a query to filter for rows where the region is 'West' and then group by product_category to find the average sale_amount.

Ans:

```
from pyspark.sql.functions import avg, col, expr

result = df.filter(
    col("Location") == "In-store"
).withColumn(
    "total_spent_numeric",
    expr("try_cast(`Total Spent` AS DOUBLE)")
).groupBy(
    "Item"
).agg(
    avg("total_spent_numeric").alias("average_spent")
)

result.show()
```

✓ 1.0s

Item	average_spent
Salad	15.398860398860398
ERROR	8.298780487804878
Tea	4.633333333333334
NULL	8.277777777777779
Sandwich	12.237960339943342
Juice	9.061224489795919
UNKNOWN	8.875
Smoothie	11.826086956521738
Coffee	6.118811881188119
Cake	9.29903536977492
Cookie	2.938311688311688

Q5: What is the difference between .na.drop() and .na.fill()? Provide a code example of filling null values in a status column with the string 'Unknown'.

Ans:

```
df_clean = df_clean.fillna({
    "Item": "Unknown",
    "Payment Method": "Unknown",
    "Location": "Unknown",
    "Quantity": 0,
    "Price Per Unit": 0.0,
    "Total Spent": 0.0
})
```

✓ 0.0s

Python

```
print("Rows Before Cleaning:", df_no_duplicates.count())
print("Rows After Handling Missing Values:", df_clean.count())
```

✓ 2.5s

Python

```
Rows Before Cleaning: 10000
Rows After Handling Missing Values: 10000
```

```
# OR drop rows with null values
# df_no_null = df.dropna()
```

✓ 0.0s

Python

1. `.na.drop()` removes rows that contain NULL values.
2. `.na.fill()` replaces NULL values with a specified value.

Q6: Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.

Ans:

```
from pyspark.sql.functions import count, col

result = df.groupBy(
    "location"
).agg(
    count("*").alias("record_count")
).filter(
    col("record_count") > 100
)

result.show()
```

✓ 1.2s

location	record_count
In-store	3017
ERROR	358
NULL	3265
Takeaway	3022
UNKNOWN	338

Q7: How does the immutability of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Ans: Spark DataFrames are immutable, which means that once a DataFrame is created, it cannot be directly modified.

When we perform data cleaning operations such as dropping a column or renaming a column, Spark creates a new DataFrame instead of changing the original DataFrame.

```
# Drop the payment_method column
df_without_payment = df.drop("payment_method")

# Rename the item column
df_renamed = df.withColumnRenamed(
    "item",
    "product"
)
```

✓ 1.2s

Q8: Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

Ans: Cafe dataset does not contain age or subscription columns.

```
Code: result = df.filter(
    (col("age").between(18, 30)) &
    (col("subscription") == "Premium")
)
result.show()
```

Q9: When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?

Ans: It is often better to handle NULL values before performing mathematical aggregations such as sum() or avg() because missing values can affect the accuracy and meaning of the result.

For example, if a price is NULL, the total revenue or average price may not correctly represent the dataset. By handling NULL values first, we can ensure that the aggregation is performed on clean and consistent data.

```
Code Example: from pyspark.sql.functions import sum
df_clean = df.na.fill({
    "total_spent": 0
})
result = df_clean.agg(
    sum("total_spent").alias("total_revenue")
)
result.show()
```

Q10: Write the code to revise a column named raw_timestamp by casting it to a TimestampType and renaming it to event_time.

```
Ans: from pyspark.sql.types import TimestampType
df_updated = df.withColumn(
    "raw_timestamp",
    col("raw_timestamp").cast(TimestampType())
).withColumnRenamed(
    "raw_timestamp",
    "event_time"
)
```

The 'raw_timestamp' column is first cast to 'TimestampType' to convert it into a timestamp data type. The column is then renamed to 'event_time' using ,withColumnRenamed()’.

Q11: Explain the "Shuffle" process that occurs during a grouping operation. Why is it considered a wide transformation?

Ans: A shuffle occurs when Spark moves data between different partitions during an operation such as groupBy().For example, if data is distributed across multiple partitions and we group by item, Spark must move records with the same item value into the same partition.This movement of data between partitions is called a shuffle.It is considered a wide transformation because one output partition may depend on data from multiple input partitions.

```
# group by item
sales_by_item = df_clean.groupBy("item").agg(
    count("*").alias("transaction_count"),
    sum("total_spent").alias("total_sales"),
    avg("total_spent").alias("average_sales")
)
sales_by_item.show()
```

✓ 1.6s

item	transaction_count	total_sales	average_sales
Salad	1148	16605.0	14.464285714285714
Tea	1089	4735.5	4.348484848484849
Sandwich	1131	12956.0	11.455349248452697
Unknown	969	8140.0	8.400412796697626
Juice	1171	9984.0	8.52604611443211
Smoothie	1096	12556.0	11.456204379562044
Coffee	1165	6784.0	5.823175965665236
Cake	1139	9933.0	8.720807726075504
Cookie	1092	3070.0	2.811355311355311

Q12: Write a code snippet that identifies and removes rows where the email column contains null values OR the username is an empty string.

Ans: Identify and remove rows where the email column contains NULL values OR the username is an empty string.

```
Code: df_clean = df.filter(
    col("email").isNull() &
    (col("username") != "")
)
```

Explanation

isNull() removes rows where email is NULL.

username != "" removes rows where the username is empty.

& means AND.

Q13: How do you use the .agg() function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

Ans:

```
summary = df_clean.select(
    count("*").alias("total_rows"),
    sum("total_spent").alias("total_sales"),
    avg("total_spent").alias("average_spent"),
    min("total_spent").alias("minimum_spent"),
    max("total_spent").alias("maximum_spent")
)

summary.show()
```

✓ 1.7s

total_rows	total_sales	average_spent	minimum_spent	maximum_spent
10000	84763.5	8.47635	0.0	25.0

Q14: In the context of cleaning a dataset, what is the risk of using inferSchema=true when your source data contains messy or inconsistent date formats?

Ans: Using inferSchema=True with messy or inconsistent date formats can cause Spark to infer the wrong data type or fail to correctly identify the date column.

For example, dates such as:

2023-01-10

10/01/2023

January 10, 2023

have different formats.

This can result in:

- Incorrect schema inference
- NULL values
- Data conversion errors
- Incorrect date-based analysis

For inconsistent date data, it is safer to explicitly define the schema or clean and parse the dates using date functions.

Q15: Write a final processing pipeline that:

1. Filters out duplicates.

2. Fills null prices with 0.
3. Groups by store_id to calculate total revenue.

Ans:

```
# clean inconsistent data
pipeline_result = (df.dropDuplicates())

for column in text_columns:
    pipeline_result = pipeline_result.withColumn(
        column,
        when(
            trim(col(column)).isin("", "ERROR", "UNKNOWN", "NULL"),
            None
        ).otherwise(trim(col(column)))
    )

for column in numeric_columns:
    pipeline_result = pipeline_result.withColumn(
        column,
        when(
            trim(col(column)).isin("", "ERROR", "UNKNOWN", "NULL"),
            None
        ).otherwise(trim(col(column)))
    )

# handle missing values
pipeline_result = pipeline_result.fillna({
    "Item": "Unknown",
    "Payment Method": "Unknown",
    "Location": "Unknown",
    "Quantity": 0,
    "Price Per Unit": 0.0,
    "Total Spent": 0.0
})

# modify schema
pipeline_result = pipeline_result.withColumn("Quantity",
    col("Quantity").cast(IntegerType())
)

pipeline_result = pipeline_result.withColumn("Price Per Unit",
    col("Price Per Unit").cast(DoubleType())
)

pipeline_result = pipeline_result.withColumn("Total Spent",
    col("Total Spent").cast(DoubleType())
)

# rename columns
for column in pipeline_result.columns:
    pipeline_result = pipeline_result.withColumnRenamed(
        column,
        column.lower().replace(" ", "_")
    )

pipeline_result.show(5)
```

✓ 9.7s

transaction_id	item	quantity	price_per_unit	total_spent	payment_method	location	transaction_date
TXN_5387411	Cake	3	3.0	0.0	Digital Wallet	Takeaway	2023-10-21
TXN_3381656	Cake	5	3.0	15.0	Cash	Takeaway	2023-10-26
TXN_6111710	Salad	3	5.0	15.0	Credit Card	In-store	2023-06-18
TXN_8715122	Cake	2	3.0	6.0	Cash	Takeaway	ERROR
TXN_1270603	Juice	4	3.0	12.0	Credit Card	Takeaway	2023-12-03

only showing top 5 rows